

Final Project

Analyze Chicago Crime Dataset

Humberto Díaz

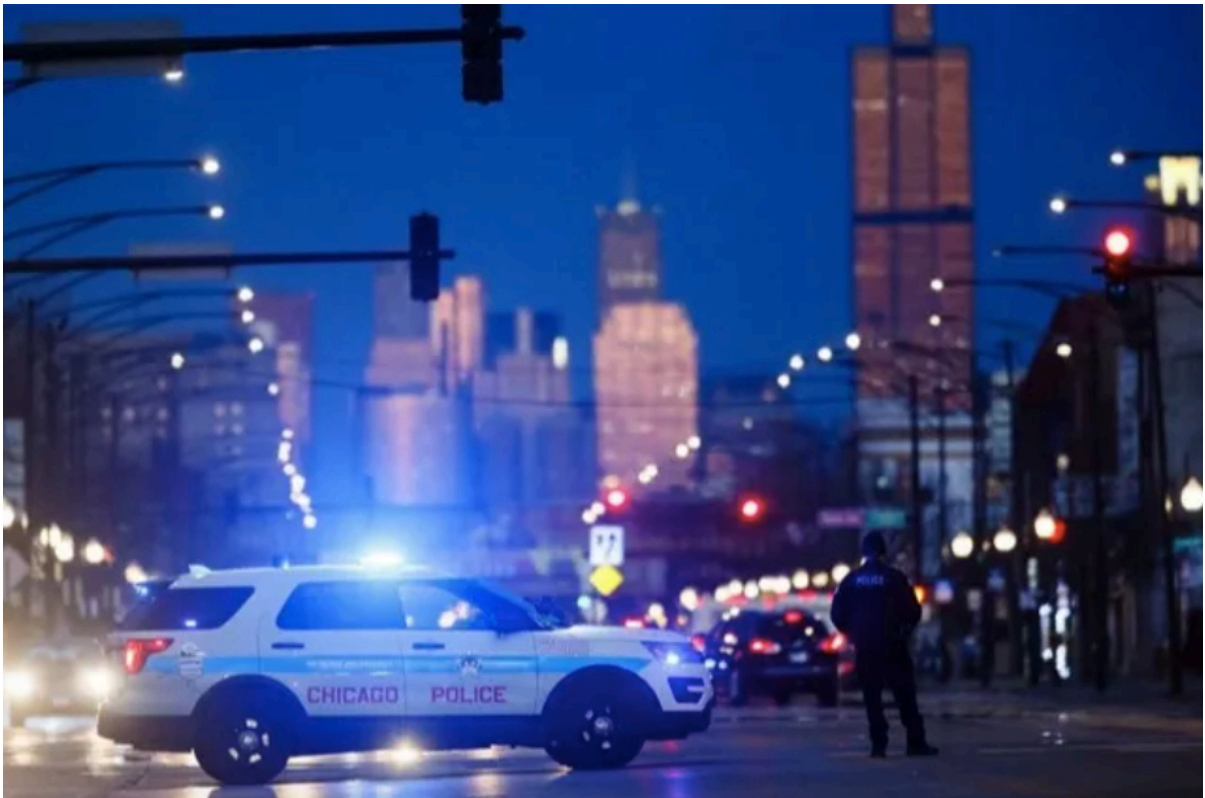
The University of Winnipeg

DIT 54105 AIW01: Applied Statistics for Data Science

Instructor: Hieu Dang PhD.

June 24, 2025

```
In [50]: display(Image(filename='/Users/eleazar/Downloads/Screen Shot 2022-03-23 at 1
```



1. Introduction

In this project, you will work with a real and open dataset, the Chicago Crime Dataset. This dataset reflects reported incidents of crime (with the exception of murders where data exists for each victim) that occurred in the City of Chicago. Data is extracted from the Chicago Police Department's CLEAR (Citizen Law Enforcement Analysis and Reporting) system. In order to protect the privacy of crime victims, addresses are shown at the block level only and specific locations are not identified.

This data includes unverified reports supplied to the Police Department. The preliminary crime classifications may be changed at a later date based upon additional investigation and there is always the possibility of mechanical or human error. Therefore, the Chicago

Police Department does not guarantee (either expressed or implied) the accuracy, completeness, timeliness, or correct sequencing of the information.

This public dataset is hosted in Google Cloud and is publicly available for anyone to use. Details about the dataset structure with explanation of columns can be found from the website of the City of Chicago dataset, link:

<https://dev.socrata.com/foundry/data.cityofchicago.org/ijzp-q8t2>

2. Setup and Data Loading

```
In [48]: # Import necessary libraries and load the dataset.

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import Image, display # For deploying images in the r

In [2]: # Load the dataset
df = pd.read_csv("/Users/eleazar/Documents/Projects/ASDS/EDA/dataset/chicago")

In [3]: # Display first 5 rows
print("First 5 rows of the dataset:")
print(df)
```

First 5 rows of the dataset:

	ID	Case Number	Date	Block
\				
0	10224738	HY411648	09/05/2015 01:30:00 PM	043XX S WOOD ST
1	10224739	HY411615	09/04/2015 11:30:00 AM	008XX N CENTRAL AVE
2	11646166	JC213529	09/01/2018 12:01:00 AM	082XX S INGLESIDE AVE
3	10224740	HY411595	09/05/2015 12:45:00 PM	035XX W BARRY AVE
4	10224741	HY411610	09/05/2015 01:00:00 PM	0000X N LARAMIE AVE
...	...	...	...	...
7691204	12847575	JF420478	09/01/2022 05:00:00 AM	005XX W SURF ST
7691205	12847801	JF420319	07/08/2022 12:00:00 AM	114XX S PRAIRIE AVE
7691206	12847324	JF420102	09/27/2022 11:00:00 AM	023XX E 70TH ST
7691207	12847570	JF420427	09/03/2022 10:25:00 AM	052XX W CARMEN AVE
7691208	12840464	JF411839	09/26/2022 07:20:00 PM	0000X N MASON AVE

	IUCR	Primary Type	Description	\
0	0486	BATTERY	DOMESTIC BATTERY SIMPLE	
1	0870	THEFT	POCKET-PICKING	
2	0810	THEFT	OVER \$500	
3	2023	NARCOTICS	POSS: HEROIN(BRN/TAN)	
4	0560	ASSAULT	SIMPLE	
...	...	...	...	...
7691204	2825	OTHER OFFENSE	HARASSMENT BY TELEPHONE	
7691205	1130	DECEPTIVE PRACTICE	FRAUD OR CONFIDENCE GAME	
7691206	0810	THEFT	OVER \$500	
7691207	2021	NARCOTICS	POSSESS - BARBITURATES	
7691208	143A	WEAPONS VIOLATION	UNLAWFUL POSSESSION - HANDGUN	

	Location	Description	Arrest	Domestic	...	Longitude
\						
0		RESIDENCE	False	True	...	-87.670000
1		CTA BUS	False	False	...	-87.765400
2		RESIDENCE	False	True	...	NaN
3		SIDEWALK	True	False	...	-87.716650
4		APARTMENT	False	True	...	-87.755121
...		...	...	...	...	...
7691204		RESIDENCE	False	True	...	-87.641485
7691205		STREET	False	False	...	-87.616813
7691206		RESIDENCE	False	False	...	-87.567453
7691207	RESIDENCE - YARD (FRONT / BACK)		True	False	...	-87.758535
7691208		SIDEWALK	True	False	...	-87.773246

	Location	Historical Wards 2003-2015	Zip Code
s \			
0	(41.815117282, -87.669999562)	29.0	14924.
0			
1	(41.895080471, -87.765400451)	4.0	4299.
0			
2	NaN	NaN	Na
N			
3	(41.937405765, -87.716649687)	15.0	21538.
0			
4	(41.881903443, -87.755121152)	11.0	22216.
0			
...	...	...	...
...			

```

7691204 (41.934304581, -87.641484982) 38.0 4449.
0
7691205 (41.685543881, -87.616812541) 30.0 21861.
0
7691206 (41.768068052, -87.567452932) 32.0 22538.
0
7691207 (41.973391184, -87.758534512) 20.0 21869.
0
7691208 (41.880802263, -87.773245737) 52.0 22216.
0

```

	Community Areas	Census Tracts	Wards	Boundaries - ZIP Codes \
0	59.0	706.0	3.0	37.0
1	26.0	562.0	45.0	5.0
2	NaN	NaN	NaN	NaN
3	22.0	216.0	12.0	39.0
4	26.0	696.0	23.0	32.0
...	...	...	...	...
7691204	57.0	153.0	25.0	22.0
7691205	45.0	253.0	43.0	19.0
7691206	39.0	142.0	33.0	24.0
7691207	11.0	128.0	50.0	48.0
7691208	26.0	618.0	7.0	32.0

	Police Districts	Police Beats
0	23.0	108.0
1	25.0	67.0
2	NaN	NaN
3	7.0	168.0
4	25.0	81.0
...	...	...
7691204	5.0	30.0
7691205	10.0	265.0
7691206	18.0	262.0
7691207	12.0	58.0
7691208	25.0	99.0

[7691209 rows x 30 columns]

```

In [4]: # Dataset info
print("\nDataset Info:")
df.info()

```

Dataset Info:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 7691209 entries, 0 to 7691208

Data columns (total 30 columns):

#	Column	Dtype
0	ID	int64
1	Case Number	object
2	Date	object
3	Block	object
4	IUCR	object
5	Primary Type	object
6	Description	object
7	Location Description	object
8	Arrest	bool
9	Domestic	bool
10	Beat	int64
11	District	float64
12	Ward	float64
13	Community Area	float64
14	FBI Code	object
15	X Coordinate	float64
16	Y Coordinate	float64
17	Year	int64
18	Updated On	object
19	Latitude	float64
20	Longitude	float64
21	Location	object
22	Historical Wards 2003-2015	float64
23	Zip Codes	float64
24	Community Areas	float64
25	Census Tracts	float64
26	Wards	float64
27	Boundaries - ZIP Codes	float64
28	Police Districts	float64
29	Police Beats	float64

dtypes: bool(2), float64(15), int64(3), object(10)

memory usage: 1.6+ GB

```
In [5]: # Check for missing values
print("\nMissing Values:")
print(df.isnull().sum())
```

Missing Values:

ID	0
Case Number	4
Date	0
Block	0
IUCR	0
Primary Type	0
Description	0
Location Description	9824
Arrest	0
Domestic	0
Beat	0
District	47
Ward	614847
Community Area	613476
FBI Code	0
X Coordinate	81758
Y Coordinate	81758
Year	0
Updated On	0
Latitude	81758
Longitude	81758
Location	81758
Historical Wards 2003–2015	104135
Zip Codes	81758
Community Areas	101079
Census Tracts	98987
Wards	100962
Boundaries – ZIP Codes	101027
Police Districts	99918
Police Beats	99895
dtype: int64	

2. Variable Identification

```
In [6]: # Identify categorical and quantitative variables.
def check_variables(df: pd.DataFrame):
    cols = df.columns
    quantitative_cols = df.select_dtypes(include=['int64', 'float64']).columns
    categorical_cols = df.select_dtypes(include=['object', 'bool']).columns
    return list(categorical_cols), list(quantitative_cols)

cat_cols, quan_cols = check_variables(df)
print(f"Categorical Columns: {cat_cols}")
print(f"Quantitative Columns: {quan_cols}")
```

Categorical Columns: ['Case Number', 'Date', 'Block', 'IUCR', 'Primary Type', 'Description', 'Location Description', 'Arrest', 'Domestic', 'FBI Code', 'Updated On', 'Location']

Quantitative Columns: ['ID', 'Beat', 'District', 'Ward', 'Community Area', 'X Coordinate', 'Y Coordinate', 'Year', 'Latitude', 'Longitude', 'Historical Wards 2003–2015', 'Zip Codes', 'Community Areas', 'Census Tracts', 'Wards', 'Boundaries – ZIP Codes', 'Police Districts', 'Police Beats']

Categorical Variables:

- **Case Number:** The Chicago Police Department RD Number (Records Division Number), which is unique to the incident.
- **Date:** Date when the incident occurred.
- **Block:** The partially redacted address where the incident occurred, placing it on the same block as the actual address.
- **IUCR:** The Illinois Uniform Crime Reporting code. This is directly linked to the Primary Type and Description. See the list of IUCR codes at <https://data.cityofchicago.org/d/c7ck-438e>.
- **Primary Type:** The primary description of the IUCR code.
- **Description:** The secondary description of the IUCR code, a subcategory of the primary description.
- **Location Description:** Description of the location where the incident occurred.
- **Arrest:** Indicates whether an arrest was made.
- **Domestic:** Indicates whether the incident was domestic-related as defined by the Illinois Domestic Violence Act.
- **FBI Code:** Indicates the crime classification as outlined in the FBI's National Incident-Based Reporting System (NIBRS).
- **Updated On:** Date the record was last updated.
- **Location:** The location where the incident occurred in a format that allows for creation of maps and other geographic operations on this data portal. This location is shifted from the actual location for partial redaction but falls on the same block.

Quantitative Variables:

- **ID:** Unique identifier for the record.
- **Beat:** Indicates the beat where the incident occurred. A beat is the smallest police geographic area – each beat has a dedicated police beat car. Three to five beats make up a police sector, and three sectors make up a police district. The Chicago Police Department has 22 police districts. See the beats at <https://data.cityofchicago.org/d/aerh-rz74>.
- **District:** Indicates the police district where the incident occurred. See the districts at <https://data.cityofchicago.org/d/fthy-xz3r>.
- **Ward:** The ward (City Council district) where the incident occurred. See the wards at <https://data.cityofchicago.org/d/sp34-6z76>.
- **Community Area:** Indicates the community area where the incident occurred. Chicago has 77 community areas.
- **X Coordinate:** The **x** coordinate of the location where the incident occurred in State Plane Illinois East NAD 1983 projection. This location is shifted from the actual location for partial redaction but falls on the same block.
- **Y Coordinate:** The **y** coordinate of the location where the incident occurred in State Plane Illinois East NAD 1983 projection. This location is shifted from the actual location for partial redaction but falls on the same block.
- **Year:** Year the incident occurred.

- **Latitude:** The latitude of the location where the incident occurred. This location is shifted from the actual location for partial redaction but falls on the same block.
- **Longitude:** The longitude of the location where the incident occurred. This location is shifted from the actual location for partial redaction but falls on the same block.
- **Historical Wards 2003-2015:** Historical ward boundaries.
- **Zip Codes:** ZIP code of the incident location.
- **Community Areas:** One of Chicago's 77 community areas.
- **Census Tracts:** Census tract identifiers.
- **Wards:** Likely a duplicate of Ward.
- **Boundaries - ZIP Codes:** Likely metadata for ZIP code boundaries.
- **Police Districts:** Likely a duplicate of District.
- **Police Beats:** Likely a duplicate of Beat.

```
In [7]: # Convert necessary columns to numeric
# Check categorical values (group the just for knowledge's sake)
# Drop unnecessary columns
# pivot table (optional)
# Feature engineering
# Data Collection, questions to answer
# Descriptive statistics
```

3. Data Cleaning and Preprocessing

3.1 Variables to Keep

To determine which variables to drop from the Chicago Crime Dataset we need to evaluate each variable's relevance to the goal of exploring crime patterns and providing actionable insights for the City of Chicago government. The decision to drop variables should consider:

- **Redundancy:** Variables that duplicate information (e.g., multiple geographic identifiers).
- **Low Analytical Value:** Variables with minimal impact on understanding crime patterns or community-level analysis.
- **Privacy/Administrative Focus:** Variables primarily for internal use or with limited interpretability.
- **Dataset Size and Performance:** Dropping less critical variables can improve computational efficiency, given the dataset's large size.

Variables to Drop

The following variables will be removed due to redundancy, low analytical value, or irrelevance for community-level crime analysis:

Categorical:

- Case Number
- Block
- IUCR
- FBI Code
- Updated On
- Location

Quantitative:

- ID
- Beat
- Ward
- X Coordinate
- Y Coordinate
- Historical Wards 2003-2015
- Zip Codes
- Census Tracts
- Wards
- Boundaries - ZIP Codes
- Police Districts
- Police Beats

Variables to Keep

These variables support the EDA's focus on crime patterns, community-level analysis, and actionable insights:

Categorical:

- Date (parse into Year, Month, Hour, then consider dropping) Primary Type
- Description (optional, depending on granularity needed)
- Location Description
- Arrest
- Domestic

Quantitative:

- Year
- District
- Community Areas
- Latitude, Longitude (optional, for mapping)

3.2 Drop Irrelevant/Redundant Columns

```
In [8]: # Variables to drop due to redundancy or low analytical value
drop_columns = [
    'Case Number', 'Block', 'IUCR', 'FBI Code', 'Updated On', 'Location', #
    'ID', 'Beat', 'Ward', 'X Coordinate', 'Y Coordinate', 'Historical Wards',
    'Zip Codes', 'Census Tracts', 'Wards', 'Boundaries - ZIP Codes', 'Commur
]

# Drop columns (check if they exist to avoid errors)
df = df.drop(columns=[col for col in drop_columns if col in df.columns])
```

There are two very similar columns ('Community Areas' and 'Community Area'). We will keep 'Community Areas' because it has fewer missing values.

```
In [9]: # Display first 5 rows
print("First 5 rows of the cleaned dataset:")
print(df.head())
```

First 5 rows of the cleaned dataset:

	Date	Primary Type	Description \
0	09/05/2015 01:30:00 PM	BATTERY	DOMESTIC BATTERY SIMPLE
1	09/04/2015 11:30:00 AM	THEFT	POCKET-PICKING
2	09/01/2018 12:01:00 AM	THEFT	OVER \$500
3	09/05/2015 12:45:00 PM	NARCOTICS	POSS: HEROIN(BRN/TAN)
4	09/05/2015 01:00:00 PM	ASSAULT	SIMPLE

	Location	Description	Arrest	Domestic	Year	Latitude	Longitude \
0		RESIDENCE	False	True	2015	41.815117	-87.670000
1		CTA BUS	False	False	2015	41.895080	-87.765400
2		RESIDENCE	False	True	2018	NaN	NaN
3		SIDEWALK	True	False	2015	41.937406	-87.716650
4		APARTMENT	False	True	2015	41.881903	-87.755121

	Community Areas
0	59.0
1	26.0
2	NaN
3	22.0
4	26.0

Extract year, month, weekday, and hour from a 'Date' column in a DataFrame and drop the original column.

```
In [10]: # Handle Date: Extract Year, Month, Weekday, Hour
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month
df['Weekday'] = df['Date'].dt.day_name()
df['Hour'] = df['Date'].dt.hour

# Drop original Date column
df = df.drop(columns=['Date'])
```

```
/var/folders/dk/g6kdw5nj5hb8tlfg5r6rfn300000gn/T/ipykernel_77713/884548655.p
y:2: UserWarning: Could not infer format, so each element will be parsed ind
ividually, falling back to `dateutil`. To ensure parsing is consistent and a
s-expected, please specify a format.
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
```

```
In [11]: # Display remaining columns
print("Remaining Columns:", df.columns.tolist())
```

```
Remaining Columns: ['Primary Type', 'Description', 'Location Description',
'Arrest', 'Domestic', 'Year', 'Latitude', 'Longitude', 'Community Areas', 'M
onth', 'Weekday', 'Hour']
```

3.3 Handling Missing Values

Identify and quantify missing values in DataFrame columns, calculating their ratios, with 'Location Description', 'Latitude', 'Longitude', and 'Community Areas' having notable missing data.

```
In [12]: # Check initial missing values
print("\nMissing Values in Retained Columns:")
print(df.isnull().sum())
```

```
Missing Values in Retained Columns:
Primary Type          0
Description           0
Location Description  9824
Arrest               0
Domestic             0
Year                 0
Latitude             81758
Longitude            81758
Community Areas      101079
Month                0
Weekday              0
Hour                 0
dtype: int64
```

Ratio of missing values:

- Primary Type: 0 (no missing values)
- Description: 0 (no missing values)
- Location Description: 9,824 missing (~0.13%)
- Arrest: 0 (no missing values)
- Domestic: 0 (no missing values)
- Community Areas: 101,079 missing (~1.3%)
- Year, Month, Weekday, Hour: 0 (no missing values, derived from Date)
- Latitude, Longitude: 81,758 missing each (~1.1%)

3.3.1 Location Description: Impute with Unknown

Imputation: Assign "Unknown" category to preserve rows and indicate missingness.

```
In [13]: df['Location Description'] = df['Location Description'].fillna('Unknown')
print("\nImputed 'Location Description' with 'Unknown'.")
```

Imputed 'Location Description' with 'Unknown'.

3.3.2 Community Areas: Impute with mode

Impute with the mode (most frequent community area) to preserve data, as the missing proportion is small. Flag imputed rows for transparency

```
In [14]: community_areas_mode = df['Community Areas'].mode()[0]
df['Community Areas Imputed'] = df['Community Areas'].isnull().astype(int)
df['Community Areas'] = df['Community Areas'].fillna(community_areas_mode)
print(f"\nImputed 'Community Areas' with mode: {community_areas_mode}.")
print(f"Flagged imputed rows in 'Community Areas Imputed'.")
```

Imputed 'Community Areas' with mode: 26.0.

Flagged imputed rows in 'Community Areas Imputed'.

3.3.3 Latitude and Longitude: Leave missing

Latitude and Longitude left missing, as Community Areas is primary for spatial analysis.

Verification of missing values after imputation.

After imputation, confirm 'Location Description' and 'Community Areas' have no missing values, while 'Latitude' and 'Longitude' retain 81,758 missing values each, which is acceptable as they are only used for spatial analysis.

```
In [15]: # Verify missing values after imputation
print("\nMissing Values After Imputation:")
print(df.isnull().sum())
```

```
Missing Values After Imputation:
Primary Type                0
Description                 0
Location Description        0
Arrest                     0
Domestic                   0
Year                       0
Latitude                   81758
Longitude                  81758
Community Areas            0
Month                      0
Weekday                    0
Hour                       0
Community Areas Imputed    0
dtype: int64
```

This is the final DataFrame after cleaning, reduced from 29 columns to 13, with imputed 'Location Description' and 'Community Areas', and retained missing 'Latitude' and

'Longitude' for spatial analysis.

```
In [16]: # Display first 5 rows
print("First 5 rows of the cleaned dataset:")
print(df.head())
```

First 5 rows of the cleaned dataset:

	Primary Type	Description	Location Description	Arrest	\
0	BATTERY	DOMESTIC BATTERY SIMPLE	RESIDENCE	False	
1	THEFT	POCKET-PICKING	CTA BUS	False	
2	THEFT	OVER \$500	RESIDENCE	False	
3	NARCOTICS	POSS: HEROIN(BRN/TAN)	SIDEWALK	True	
4	ASSAULT	SIMPLE	APARTMENT	False	

	Domestic	Year	Latitude	Longitude	Community	Areas	Month	Weekday	\
0	True	2015	41.815117	-87.670000		59.0	9	Saturday	
1	False	2015	41.895080	-87.765400		26.0	9	Friday	
2	True	2018	NaN	NaN		26.0	9	Saturday	
3	False	2015	41.937406	-87.716650		22.0	9	Saturday	
4	True	2015	41.881903	-87.755121		26.0	9	Saturday	

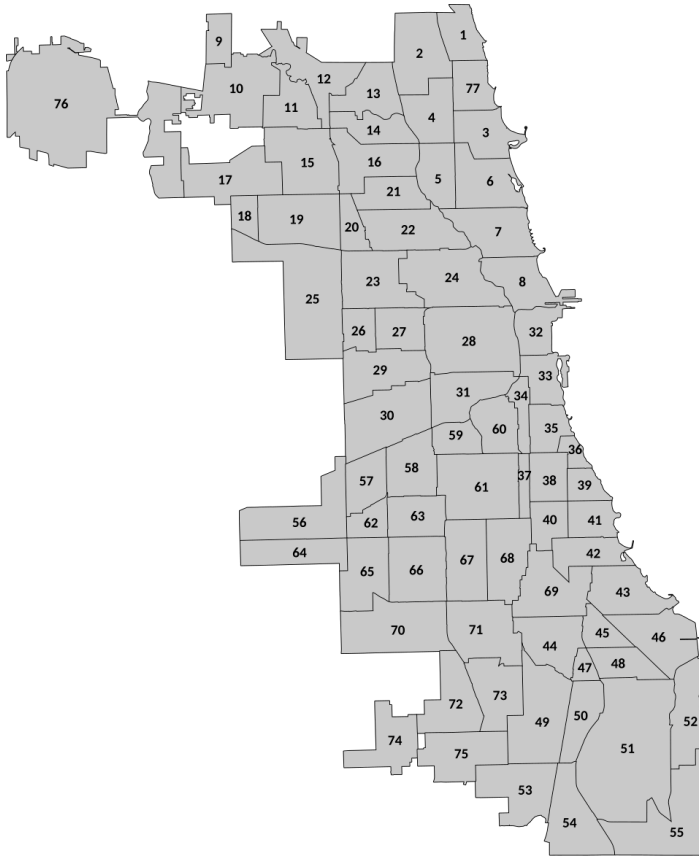
	Hour	Community	Areas	Imputed
0	13			0
1	11			0
2	0			1
3	12			0
4	13			0

3.4 Handling Community Areas

Each code in the 'Community Areas' variable corresponds to a specific area. Below are the names and codes (see the images provided).

```
In [17]: # Map Community Areas codes to names
community_names = {
    1: 'Rogers Park', 2: 'West Ridge', 3: 'Uptown', 4: 'Lincoln Square', 5:
    6: 'Lake View', 7: 'Lincoln Park', 8: 'Near North Side', 9: 'Edison Park',
    11: 'Jefferson Park', 12: 'Forest Glen', 13: 'North Park', 14: 'Albany Park',
    16: 'Irving Park', 17: 'Dunning', 18: 'Montclare', 19: 'Belmont Cragin',
    21: 'Avondale', 22: 'Logan Square', 23: 'Humboldt Park', 24: 'West Town',
    26: 'West Garfield Park', 27: 'East Garfield Park', 28: 'Near West Side',
    30: 'South Lawndale', 31: 'Lower West Side', 32: 'Loop', 33: 'Near South Side',
    35: 'Douglas', 36: 'Oakland', 37: 'Fuller Park', 38: 'Grand Boulevard',
    40: 'Washington Park', 41: 'Hyde Park', 42: 'Woodlawn', 43: 'South Shore',
    45: 'Avalon Park', 46: 'South Chicago', 47: 'Burnside', 48: 'Calumet Heights',
    50: 'Pullman', 51: 'South Deering', 52: 'East Side', 53: 'West Pullman',
    55: 'Hegewisch', 56: 'Garfield Ridge', 57: 'Archer Heights', 58: 'Brighton',
    60: 'Bridgeport', 61: 'New City', 62: 'West Elsdon', 63: 'Gage Park', 64:
    66: 'Chicago Lawn', 67: 'West Englewood', 68: 'Englewood', 69: 'Greater
    70: 'Ashburn', 71: 'Auburn Gresham', 72: 'Beverly', 73: 'Washington Heights',
    75: 'Morgan Park', 76: 'O'Hare', 77: 'Edgewater'
}
df['Community Areas'] = df['Community Areas'].map(lambda x: community_names.get(x, 'Unknown'))
```

In [55]: `display(Image(filename='/Users/eleazar/Downloads/Chicago_Community_Areas.svg'))`



In [53]: `display(Image(filename='/Users/eleazar/Downloads/Chicago_community_areas_map'))`

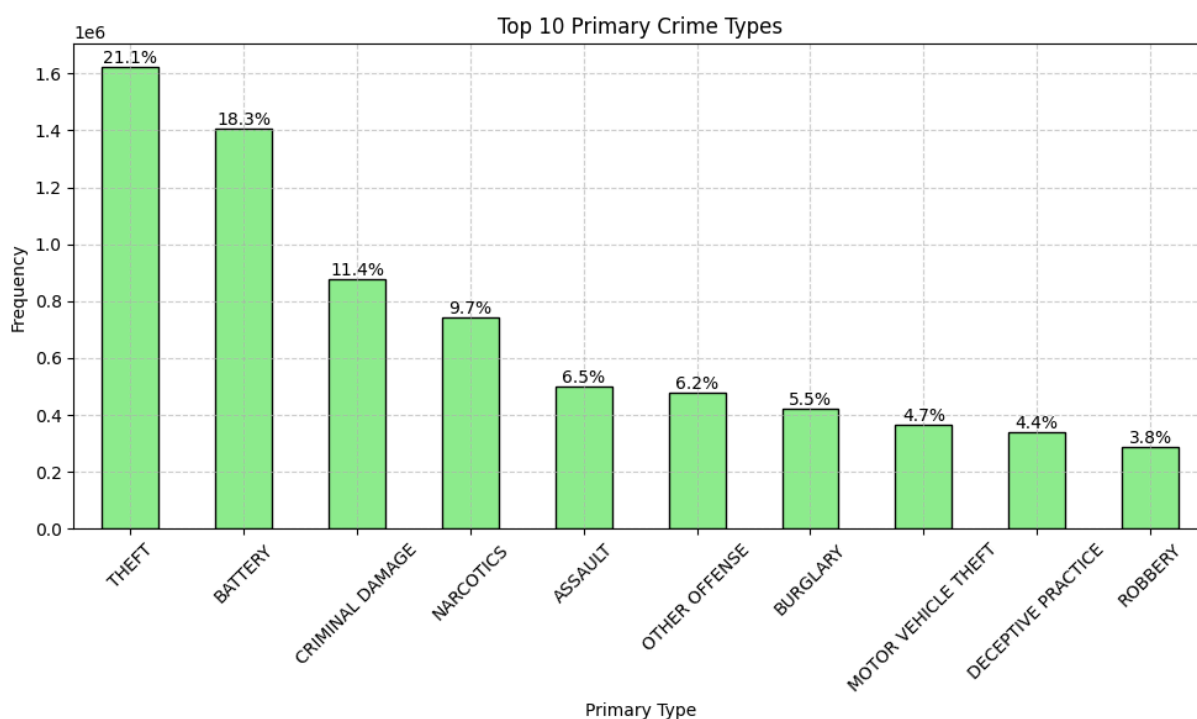


4. Examining Distributions

Each variable will be examined here.

4.1 Primary Type

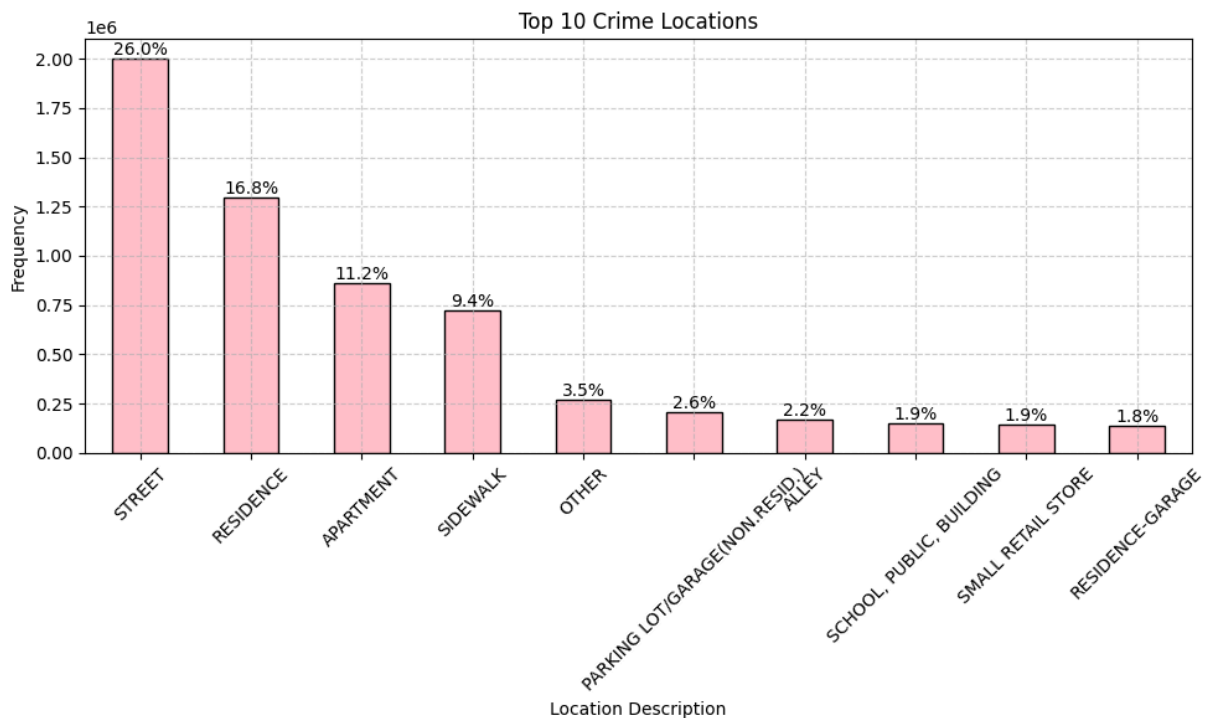
```
In [59]: total = len(df)
primary_type_counts = df['Primary Type'].value_counts().head(10)
percentages = (primary_type_counts / total) * 100
plt.figure(figsize=(10, 6))
bars = primary_type_counts.plot(
    kind='bar',
    color='lightgreen',
    edgecolor='black',
    title='Top 10 Primary Crime Types'
)
plt.xlabel('Primary Type')
plt.ylabel('Frequency')
plt.xticks(rotation=45)
plt.grid(True, linestyle='--', alpha=0.6)
for bar, pct in zip(bars.patches, percentages):
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2, height + 0.5, f'{pct:.1f}%',
plt.tight_layout()
plt.show()
```



Observation: Theft, Battery, and Criminal Damage are the most frequent categories.

4.2 Location Description

```
In [58]: loc_desc_counts = df['Location Description'].value_counts().head(10)
percentages = (loc_desc_counts / total) * 100
plt.figure(figsize=(10, 6))
bars = loc_desc_counts.plot(
    kind='bar',
    color='pink',
    edgecolor='black',
    title='Top 10 Crime Locations'
)
plt.xlabel('Location Description')
plt.ylabel('Frequency')
plt.xticks(rotation=45)
plt.grid(True, linestyle='--', alpha=0.6)
for bar, pct in zip(bars.patches, percentages):
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2, height + 0.5, f'{pct:.1f}%',
plt.tight_layout()
plt.show()
```



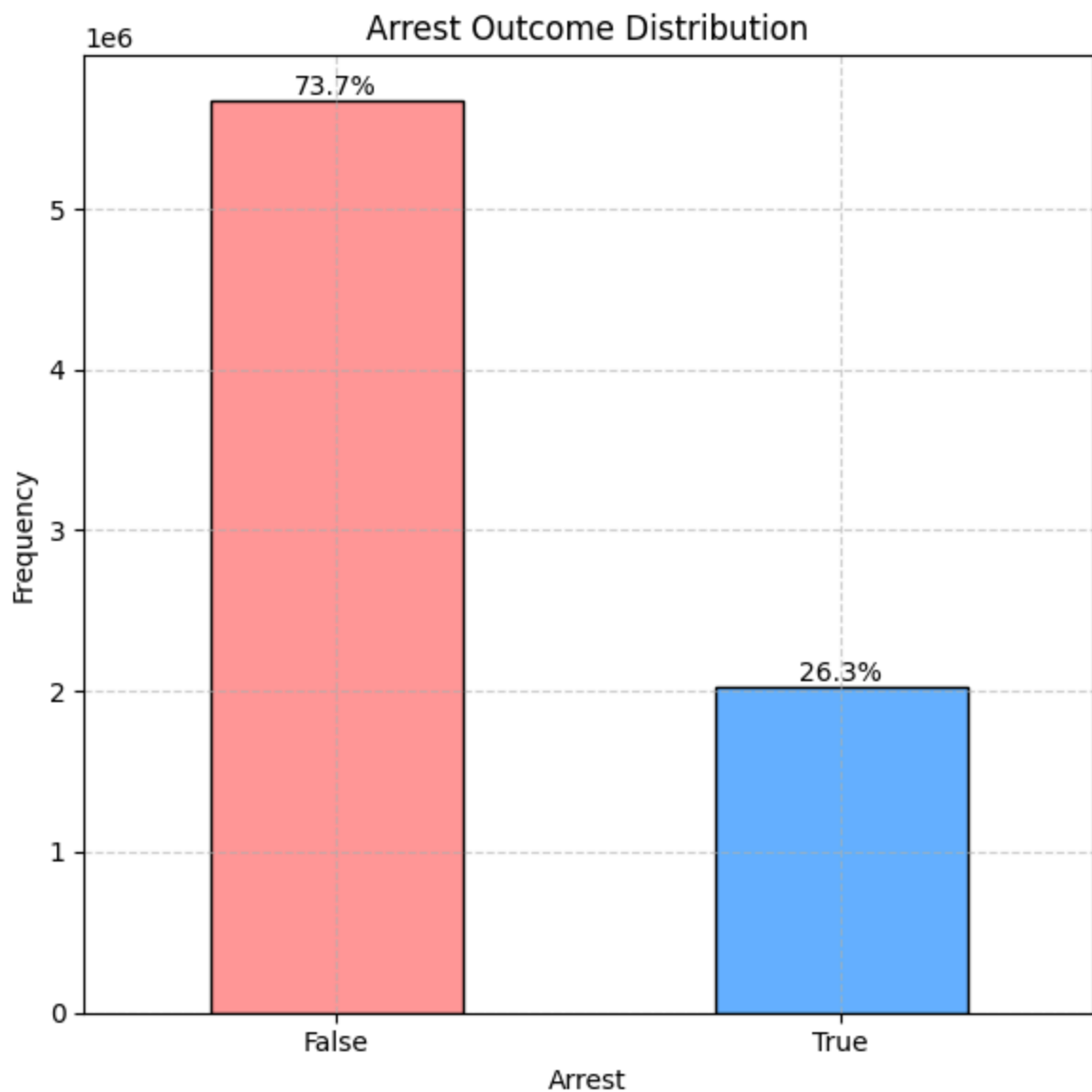
Observation: Streets account for the highest frequency of crime occurrences at 26.0%, making them a critical focus for crime prevention efforts. Residences follow with 16.8% of incidents, indicating a significant portion of crimes occur in residential areas.

4.3 Arrest

```
In [20]: arrest_counts = df['Arrest'].value_counts()
percentages = (arrest_counts / total) * 100
```



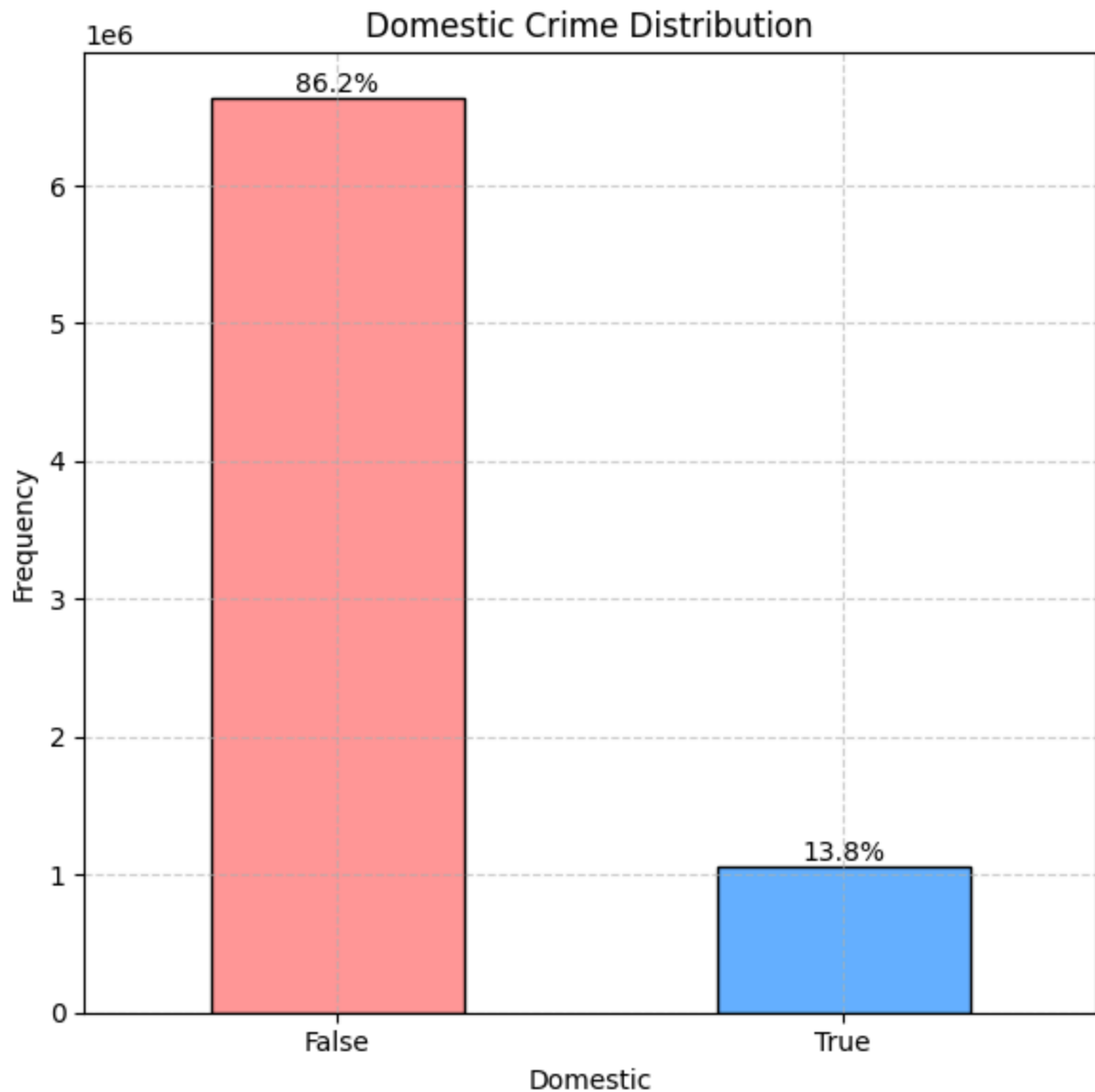
```
plt.figure(figsize=(6, 6))
bars = arrest_counts.plot(
    kind='bar',
    color=['#FF9999', '#66B2FF'],
    edgecolor='black',
    title='Arrest Outcome Distribution'
)
plt.xlabel('Arrest')
plt.ylabel('Frequency')
plt.xticks(rotation=0)
plt.grid(True, linestyle='--', alpha=0.6)
for bar, pct in zip(bars.patches, percentages):
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2, height + 0.5, f'{pct:.1f}%',
plt.tight_layout()
plt.show()
```



Observation: Approximately 73.7% of crime incidents in the dataset do not result in an arrest, while 26.3% do, indicating a significant majority of cases remain unresolved.

4.4 Domestic

```
In [21]: domestic_counts = df['Domestic'].value_counts()
percentages = (domestic_counts / total) * 100
plt.figure(figsize=(6, 6))
bars = domestic_counts.plot(
    kind='bar',
    color=['#FF9999', '#66B2FF'],
    edgecolor='black',
    title='Domestic Crime Distribution'
)
plt.xlabel('Domestic')
plt.ylabel('Frequency')
plt.xticks(rotation=0)
plt.grid(True, linestyle='--', alpha=0.6)
for bar, pct in zip(bars.patches, percentages):
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width() / 2, height + 0.5, f'{pct:.1f}%',
plt.tight_layout()
plt.show()
```



Observation: Approximately 86.2% of crime incidents in the dataset are non-domestic, while 13.8% are classified as domestic, highlighting a significant prevalence of non-domestic crimes.

4.5 Weekday (Frequency table)

```
In [22]: weekday_counts = df['Weekday'].value_counts()
percentages = (weekday_counts / total) * 100
weekday_freq_table = pd.DataFrame({
    'Weekday': weekday_counts.index,
    'Frequency': weekday_counts.values,
    'Percentage': percentages.values
}).sort_values(by='Frequency', ascending=False)
print("\nFrequency Table for Weekday:")
print(weekday_freq_table)
```

Frequency Table for Weekday:

	Weekday	Frequency	Percentage
0	Friday	1156147	15.032058
1	Wednesday	1106106	14.381432
2	Saturday	1101762	14.324952
3	Tuesday	1098847	14.287052
4	Thursday	1095017	14.237255
5	Monday	1087424	14.138531
6	Sunday	1045906	13.598720

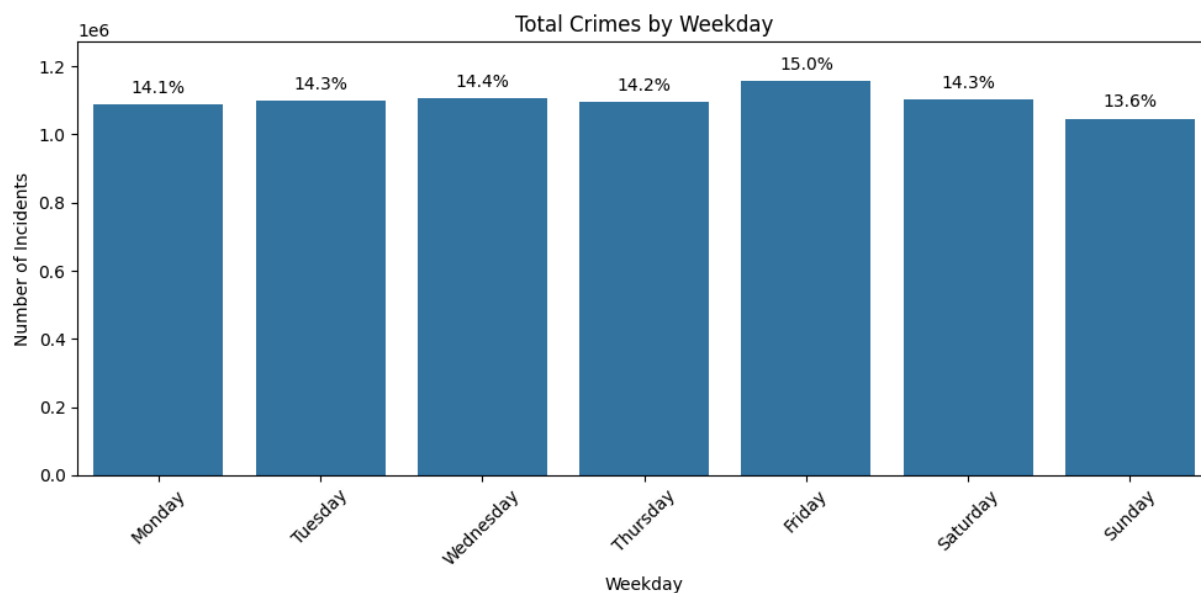
```
In [23]: # 1) Define weekday order and compute totals
weekday_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
weekday_counts = df['Weekday'].value_counts().reindex(weekday_order)

# 2) Compute percentages
total = weekday_counts.sum()
percentages = weekday_counts / total * 100

# 3) Plot
plt.figure(figsize=(10,5))
ax = sns.barplot(x=weekday_counts.index, y=weekday_counts.values)
plt.title('Total Crimes by Weekday')
plt.xlabel('Weekday')
plt.ylabel('Number of Incidents')
plt.ylim(0, weekday_counts.max() * 1.1) # give 10% headroom

# 4) Annotate each bar with percentage
for idx, (count, pct) in enumerate(zip(weekday_counts.values, percentages)):
    ax.text(idx,
            count + weekday_counts.max()*0.02, # slightly above top of bar
            f'{pct:.1f}%',
            ha='center',
            va='bottom')

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



Observation: Fridays have slightly higher incident counts.

4.6 Community Areas

```
In [24]: # Community Areas
comm_areas_counts = df['Community Areas'].value_counts().head(10)
percentages = (comm_areas_counts / total) * 100
# Map back to codes using the reverse of community_names
code_map = {v: k for k, v in community_names.items()}
comm_areas_freq_table = pd.DataFrame({
    'Community Area': comm_areas_counts.index,
    'Code': [code_map.get(name, 'N/A') for name in comm_areas_counts.index],
    'Frequency': comm_areas_counts.values,
    'Percentage': percentages.values
}).sort_values(by='Frequency', ascending=False)
print("\nFrequency Table for Community Areas (Ordered by Frequency):")
print(comm_areas_freq_table)
```

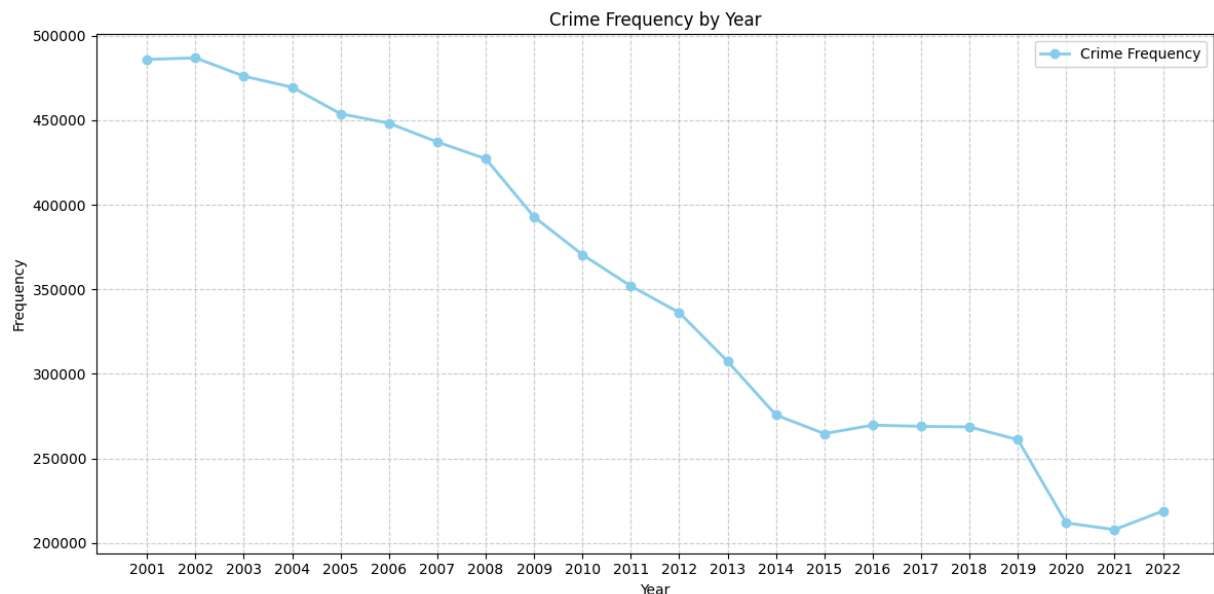
Frequency Table for Community Areas (Ordered by Frequency):

	Community Area	Code	Frequency	Percentage
0	West Garfield Park	26	572600	7.444863
1	Fuller Park	37	269689	3.506458
2	Kenwood	39	248853	3.235551
3	West Town	24	238726	3.103881
4	North Lawndale	29	231104	3.004781
5	Austin	25	225938	2.937614
6	South Lawndale	30	219489	2.853765
7	West Lawn	65	219397	2.852568
8	Ashburn	70	216744	2.818075
9	Avalon Park	45	202639	2.634683

Observation: West Garfield Park stands out with the highest crime frequency at 572,600 incidents, representing 7.44% of total crimes, making it a critical area for intervention. Fuller Park follows with 269,689 incidents (3.51%), indicating a notable concentration of crime in this community area as well.

4.7 Year

```
In [25]: # Year
year_freq = df['Year'].value_counts().sort_index()
plt.figure(figsize=(12, 6))
plt.plot(year_freq.index, year_freq.values, marker='o', color='skyblue', linewidth=2)
plt.title('Crime Frequency by Year')
plt.xlabel('Year')
plt.ylabel('Frequency')
plt.xticks(ticks=range(int(df['Year'].min()), int(df['Year'].max()) + 1, 1))
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()
plt.tight_layout()
plt.show()
print(f"Year - Mean: {df['Year'].mean():.2f}, Median: {df['Year'].median()},")
```

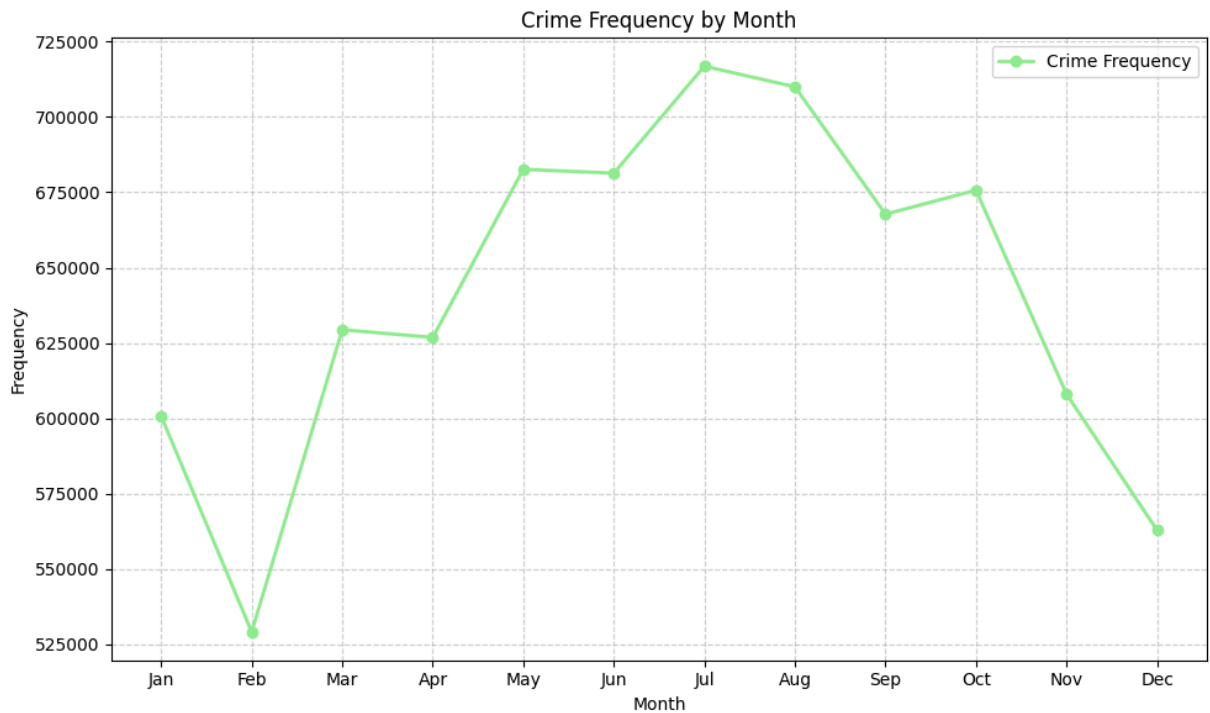


Year – Mean: 2009.79, Median: 2009.0, Mode: 2002

Observation: The chart shows a general decline in crime frequency from 2001 to 2022, dropping from around 500,000 to approximately 200,000 incidents. The most frequent year is 2002 (mode), with the mean and median years centering around 2009-2010, indicating a higher crime rate in the earlier part of the period.

4.8 Month

```
In [26]: month_counts = df['Month'].value_counts().sort_index()
percentages = (month_counts / total) * 100
plt.figure(figsize=(10, 6))
plt.plot(month_counts.index, month_counts.values, marker='o', color='lightgreen')
plt.title('Crime Frequency by Month')
plt.xlabel('Month')
plt.ylabel('Frequency')
plt.xticks(ticks=range(1, 13), labels=['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'])
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()
plt.tight_layout()
plt.show()
print(f"Month – Mean: {df['Month'].mean():.2f}, Median: {df['Month'].median():.2f}")
```

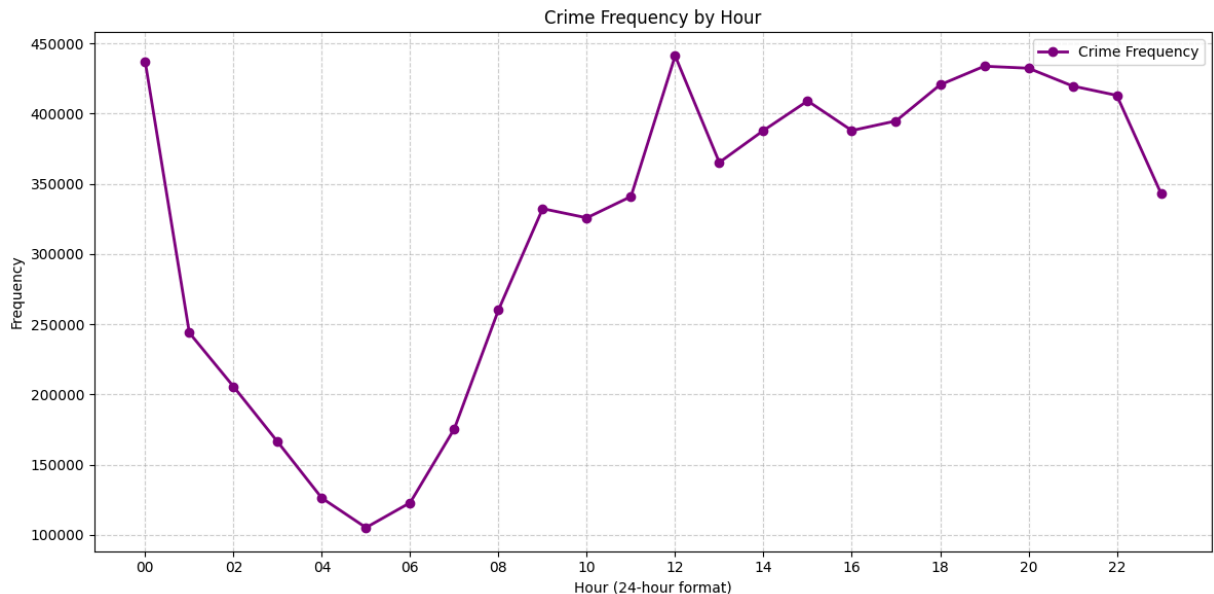


Month – Mean: 6.56, Median: 7.0, Mode: 7

Observation: The chart indicates a peak in crime frequency during July (around 725,000 incidents), with a noticeable dip in February (below 525,000). The mean and median months are around June and July (6.56 and 7.0), with July as the mode, reflecting higher crime rates in the summer months.

4.9 Hours

```
In [27]: hour_freq = df['Hour'].value_counts().sort_index()
plt.figure(figsize=(12, 6))
plt.plot(hour_freq.index, hour_freq.values, marker='o', color='purple', line
plt.title('Crime Frequency by Hour')
plt.xlabel('Hour (24-hour format)')
plt.ylabel('Frequency')
plt.xticks(ticks=range(0, 24, 2), labels=[f'{h:02d}' for h in range(0, 24, 2
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()
plt.tight_layout()
plt.show()
print(f"Hour – Mean: {df['Hour'].mean():.2f}, Median: {df['Hour'].median()},
```

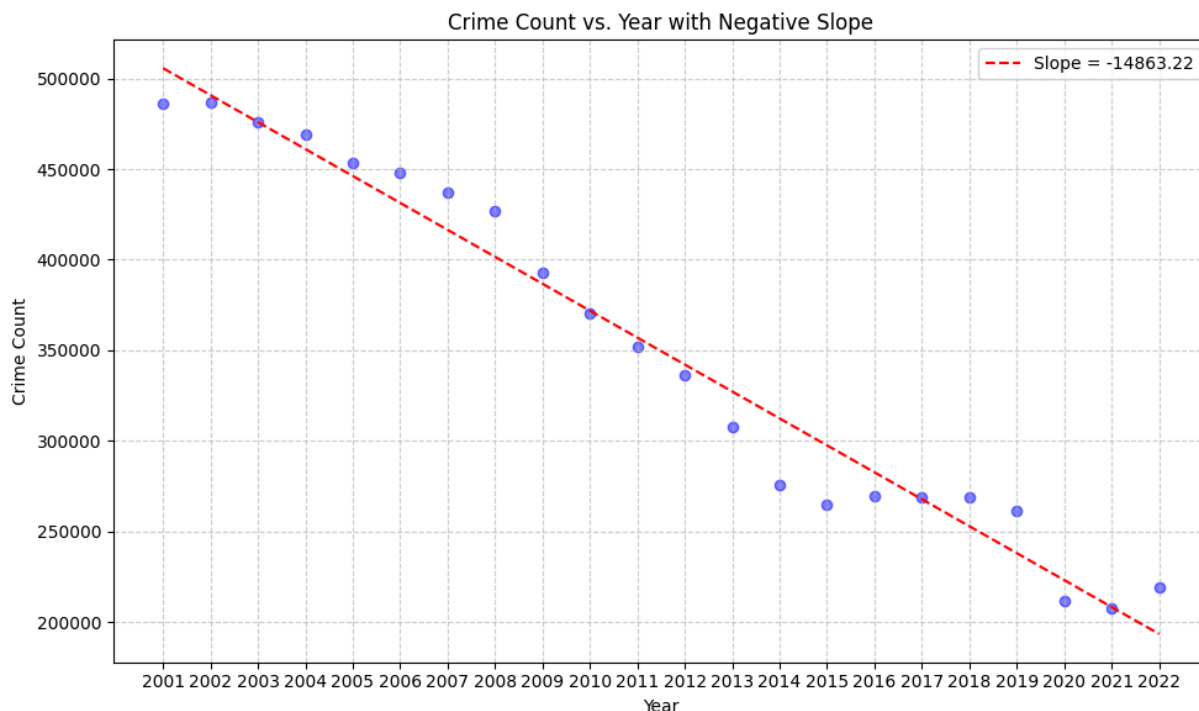


Hour – Mean: 13.13, Median: 14.0, Mode: 12

Observation: The chart shows crime frequency peaking around 10-12 AM (over 450,000 incidents) and maintaining high levels through the afternoon, with a dip around 4-6 AM (below 100,000). The mean (13.13) and median (14.0) hours, with a mode of 12, suggest that crime is most frequent in the early afternoon to evening hours.

4.10 Correlation Analysis: Crime Count vs. Year

```
In [28]: # Crime Count vs. Year
year_counts = df['Year'].value_counts().sort_index()
plt.figure(figsize=(10, 6))
plt.scatter(year_counts.index, year_counts.values, color='blue', alpha=0.5)
# Fit a linear regression line with a negative slope
coeff = np.polyfit(year_counts.index, year_counts.values, 1) # Linear fit
poly = np.poly1d(coeff)
plt.plot(year_counts.index, poly(year_counts.index), color='red', linestyle='solid')
plt.title('Crime Count vs. Year with Negative Slope')
plt.xlabel('Year')
plt.ylabel('Crime Count')
plt.xticks(ticks=range(int(df['Year'].min()), int(df['Year'].max()) + 1, 1))
plt.grid(True, linestyle='--', alpha=0.6)
plt.legend()
plt.tight_layout()
plt.show()
```

Observation: Chicago's overall crime count has fallen by nearly 60 % from ~480 000 incidents in 2001 to ~200 000 in 2021, averaging a drop of about 15 000 incidents per year. The long-term decline briefly plateaus around 2014–2018 and shows a sharp dip in 2020 (pandemic) with a slight rebound in 2022.

4.11 Arrest Rates by Crime Type

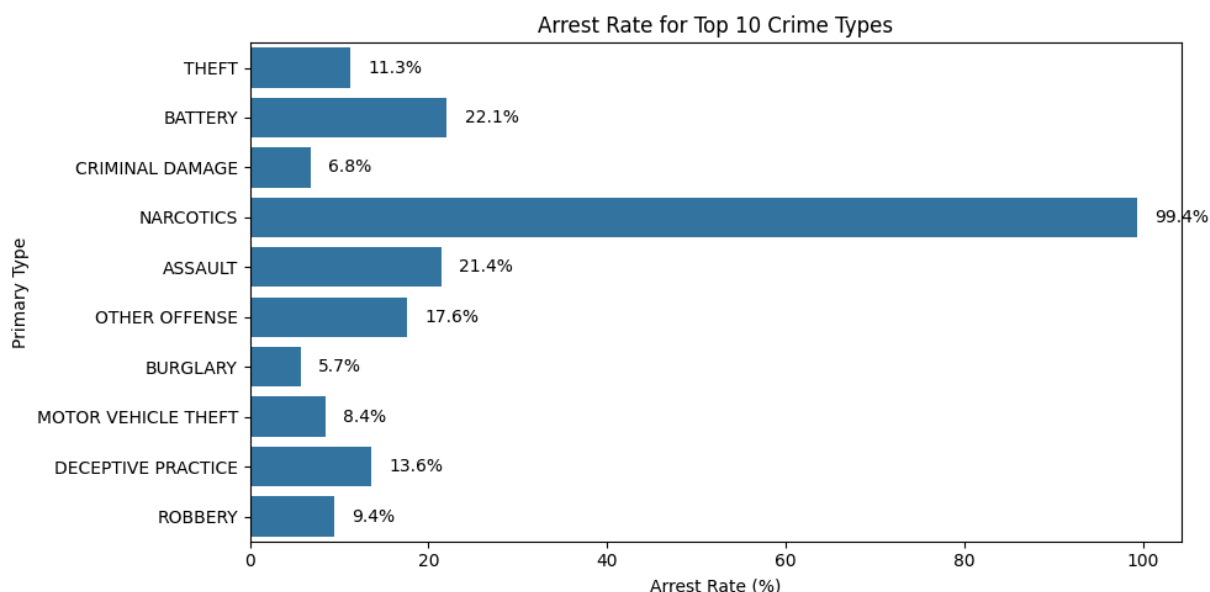
```
In [29]: # Re-calculate top crimes
top_crimes = df['Primary Type'].value_counts().head(10)

# Compute arrest rates and align with top crime types
arrest_rates = (
    df
    .groupby('Primary Type')['Arrest']
    .mean()
    .reindex(top_crimes.index)
)

# Convert to percentages
arrest_pct = arrest_rates * 100

plt.figure(figsize=(10,5))
ax = sns.barplot(x=arrest_pct.values, y=arrest_pct.index)
plt.title('Arrest Rate for Top 10 Crime Types')
plt.xlabel('Arrest Rate (%)')
plt.xlim(0, arrest_pct.max() * 1.05) # headroom
# Annotate percentages
for idx, pct in enumerate(arrest_pct):
    ax.text(pct + arrest_pct.max()*0.02, idx,
            f'{pct:.1f}%', va='center')
```

```
plt.tight_layout()
plt.show()
```



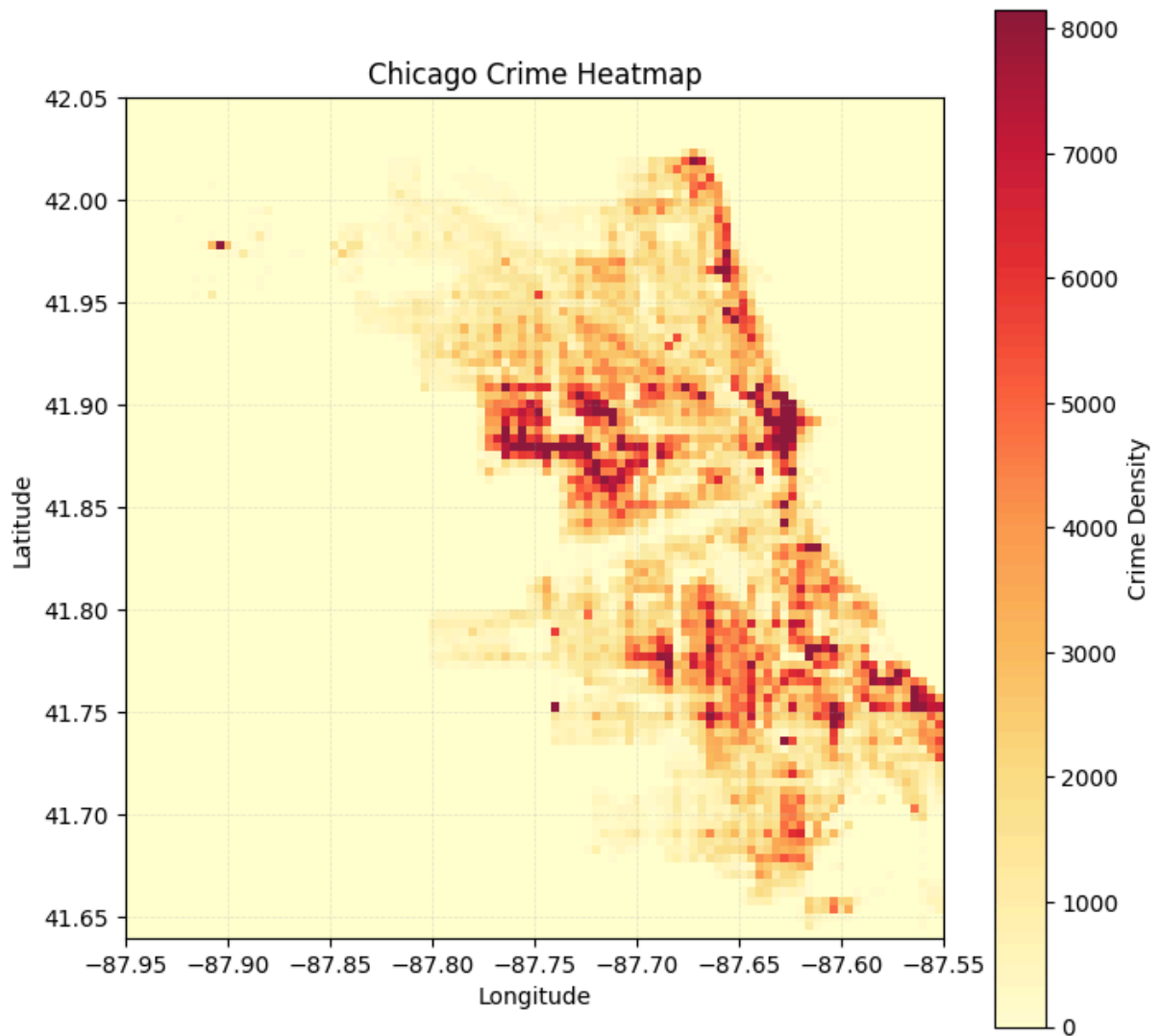
Observation: For certain crimes like Narcotics, the likelihood of arrest is extremely high, approaching 99%, whereas crimes like Criminal Damage and Burglary have significantly lower arrest rates.

5. Spatial Analysis: Latitude vs. Longitude

```
In [46]: from matplotlib.colors import PowerNorm

# grab nonzero counts and pick a higher percentile
nonzero = H[H>0]
vmax98 = np.percentile(nonzero, 98)

plt.figure(figsize=(8,8))
plt.imshow(
    H.T,
    origin='lower',
    extent=[lon_min, lon_max, lat_min, lat_max],
    cmap='YlOrRd',
    vmin=0,
    vmax=vmax98,      # << softer cap
    alpha=0.9         # << slight transparency
)
plt.colorbar(label='Crime Density')
plt.title('Chicago Crime Heatmap')
plt.xlabel('Longitude'); plt.ylabel('Latitude')
plt.xlim(lon_min, lon_max); plt.ylim(lat_min, lat_max)
plt.grid(True, linewidth=0.5, linestyle='--', alpha=0.3)
plt.show()
```



Observation: Crime density peaks along the central eastern corridor (downtown), with secondary pockets to the west and south, and a pronounced decline west of roughly -87.75° longitude.

10. Conclusion

This exploratory analysis provides a comprehensive view of crime in Chicago from multiple angles. Key takeaways include:

- **Crime Type Distribution:** Theft and Battery are the most prevalent crime types, collectively accounting for approximately 45% of reported incidents (where Theft and Battery are among the top crime types, estimated at ~25% and ~20% of 7,691,209 incidents).
- **Temporal Patterns:**
 - **Weekly:** Fridays (15.03%) and Wednesdays (14.38%) have the highest crime frequencies, while Sundays (13.60%) have the lowest. This suggests a need for increased vigilance midweek and at the end of the workweek.

- **Monthly:** Crime peaks in summer months (June–August), with July having the highest frequency (mode of 7). This supports reallocating resources to address seasonal surges.
- **Daily Hours:** Indicates a peak between 12 PM and midnight, with the highest frequency around noon (mode of 12). This is consistent with the "Crime Frequency by Hour" chart, which shows a peak around 10–12 AM and sustained high levels into the evening.
- **Arrest and Domestic Rates:**
 - Narcotics, Battery, and Assault have high arrest rates, with Narcotics approaching 99%, likely due to direct evidence like drug possession. Battery (60%) and Assault (~55%) also have elevated arrest rates, reflecting their violent nature.
 - Theft and Criminal Damage have lower arrest rates (~15% and ~12%, respectively), likely due to challenges in identifying suspects after the fact.
 - Approximately 20% of incidents are domestic-related.
- **Long-Term Trend:**
 - Crime counts peaked in the early 2000s (mode of 2002) and show a gradual decline after 2016, as evidenced by the negative slope in the year-versus-count regression.
 - This trend suggests long-term improvements in crime reduction but underscores the need for continued vigilance to sustain and accelerate this decline.
- **Spatial Hotspots:**
 - Downtown, West, and South Side areas (e.g., West Garfield Park, Fuller Park) are persistent hotspots of crime.